41. Robust Bash: errexit, nounset, traps

41. Robust Bash: errexit, nounset, traps

Phase: Phase 5 — Shell scripting
Estimated time: 1–2 days
Slug: `41-bash-robust`

Learning objectives

- Use traps for cleanup
- Handle signals in scripts
- Create temp dirs safely
- Write scripts that fail loudly

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

A script that works on your laptop when everything goes right is a demo. A production script must also behave when a command fails, the user presses Ctrl-C, the disk fills, or the same job starts twice. Robust Bash means failing loudly, cleaning up after itself, and never leaving half-written files or stale lock files. The most important tool for cleanup is `trap ... EXIT`, which runs a handler whenever the script ends, however it ends.

Core concepts

Feature	Purpose
`set -e`	Exit when a command fails (with caveats)
`set -u`	Error on unset variables
`set -o pipefail`	Pipeline fails if any stage fails
`trap CMD EXIT`	Run `CMD` when the shell exits (success, error, or exit call)
`trap CMD INT TERM`	Run on Ctrl-C or kill
`mktemp`	Create unique temp file/dir safely
`flock`	Exclusive lock so only one instance runs
`local`	Function-scoped variable
quoting	`"$var"` prevents word splitting and globbing

The cleanup phrase to remember: **trap EXIT**.

```
 start --> setup (mktemp, lock) --> work --> end
                 |                    |       |
                 +---- any exit path -+-------+--> trap handler runs: rm temp,
release lock
                       (error, exit, Ctrl-C, normal)
```

The strict-mode header most scripts should start with:

```
#!/usr/bin/env bash
set -Eeuo pipefail
IFS=$'\n\t'
```

Worked examples

Temp file with guaranteed cleanup:

```
#!/usr/bin/env bash
set -Eeuo pipefail

tmp=$(mktemp)
cleanup() { rm -f "$tmp"; }
trap cleanup EXIT

curl -fsS https://example.com > "$tmp"
wc -c < "$tmp"
```

Even if `curl` fails and the script aborts, `cleanup` runs and the file is gone.

Handle signals explicitly and keep the exit code:

```
cleanup() { rc=$?; rm -rf "$workdir"; exit "$rc"; }
workdir=$(mktemp -d)
trap cleanup EXIT
trap 'echo "interrupted" >&2; exit 130' INT TERM
```

Print line number and command on error:

```
trap 'echo "error at line $LINENO: $BASH_COMMAND" >&2' ERR
```

Single instance with `flock`:

```
exec 9>/var/lock/myjob.lock
flock -n 9 || { echo "already running" >&2; exit 1; }
```

Argument validation and usage:

```
usage() { echo "usage: $0 SRC DST" >&2; exit 2; }
[[ $# -eq 2 ]] || usage
src=$1 dst=$2
[[ -r $src ]] || { echo "cannot read: $src" >&2; exit 1; }
```

Atomic file replace (readers never see half a file):

```
tmp=$(mktemp "$dst.XXXXXX")
trap 'rm -f "$tmp"' EXIT
generate_report > "$tmp"
mv -f "$tmp" "$dst"
```

Retry with backoff:

```
retry() {
  local n=0 max=5 delay=1
  until "$@"; do
    n=$((n+1)); [[ $n -ge $max ]] && return 1
    sleep "$delay"; delay=$((delay*2))
  done
}
retry curl -fsS https://example.com/health
```

Careful with `set -e`:

```
set -e
grep -q foo file || true        # allow "not found"
if ! cmd; then echo "failed"; fi
# `set -e` is ignored inside if/while conditions and left side of && and ||
```

Lint and debug:

```
shellcheck script.sh
bash -n script.sh        # syntax check
bash -x script.sh        # trace
PS4='+ ${BASH_SOURCE}:${LINENO}: ' bash -x script.sh
```

Mini-lab

1. Write a script that creates a temp directory, writes files in it, and then calls `false`. With and without `trap ... EXIT`, check if the directory remains in `/tmp`.
2. Add `INT` handling and press Ctrl-C during a `sleep 30`; confirm cleanup.
3. Use `flock` and launch the script twice in two terminals.
4. Run `shellcheck` on an old script of yours and fix three warnings.
5. Turn on `pipefail` and test: `false | true; echo $?` with and without it.
6. Write the phrase for this module's exercise: trap EXIT.

Decision guide / failure modes

Situation	Use
Temp files	`mktemp` + `trap ... EXIT`
Cron job overlapping itself	`flock -n`
Unsafe user input in paths	quote and validate; use `--` before args
Needs structured data or complex logic	switch to Python/Go
Long script	functions, `main "$@"`, shellcheck in CI

Failure modes:

- A later `trap` replaces an earlier one for the same signal; compose them in one function.
- `set -e` surprises: it does not trigger inside conditions or in commands before `||`. Explicit checks beat magic.
- Unquoted variables with spaces turn `rm -rf $dir/*` into disaster; always quote and use `${var:?}` to refuse empty values.
- `kill -9` bypasses traps; SIGKILL cannot be caught. Design stale-lock recovery.
- Cleanup that itself fails; keep handlers tiny and use `|| true`.
- Relying on `cd` succeeding: use `cd "$dir" || exit 1`.

Common mistakes

- rm -rf unprotected tmp
- Ignoring ERR traps for critical scripts
- set +e hiding failures permanently

Exercises

Work in `exercises/41-bash-robust/`. Read that folder's `README.md` fully before coding.

```
cd exercises/41-bash-robust
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `cleanup_phrase` in `exercise.sh` so it prints the two-word Bash idiom that guarantees a cleanup handler runs when a script ends by any route. Print the exact phrase with a single space, no punctuation, and the signal name in capital letters. Then write a short script that creates a temp file, fails with `false`, and use `trap` so the temp file is removed; verify it is gone afterwards. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/41-bash-robust/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What does trap EXIT do?
2. mktemp why?
3. nounset catches what?

Further reading

- https://mywiki.wooledge.org/SignalTrap